

Linked List Skeleton 구현 스터디 가이드

문서 목적

이 문서는 `linked_list_skeleton.py` 를 직접 구현해 보는 스터디를 위한 가이드이다.

`linked_list.py` 는 완성된 정답 코드이고, `linked_list_skeleton.py` 는 직접 채워 넣어야 하는 실습 코드이다..

스터디에서는 `linked_list_skeleton.py` 의 TODO를 하나씩 구현하면서 단일 링크드리스트가 어떻게 동작하는지 이해하는 것을 목표로 한다.

파일 구성

```
LinkedList/  
├── linked_list_skeleton.py  
└── main.py
```

- `linked_list.py`: 링크드리스트 정답 코드
- `linked_list.skeleton.py`: 직접 구현할 skeleton 코드
- `main.py`: 터미널에서 링크드리스트를 실행하고 테스트하는 코드

링크드리스트 기본 개념

링크드리스트는 여러 개의 노드가 연결된 자료구조이다.

각 노드는 다음 두 가지 정보를 가진다.

- 실제 데이터 값
- 다음 노드를 가리키는 참조

배열이나 리스트처럼 인덱스로 바로 접근하는 구조가 아니라, 첫 번째 노드부터 다음 노드를 따라가며 원하는 위치를 찾아야 한다.

예를 들어 다음과 같은 링크드리스트가 있다고 해보자.

```
HEAD -> [A] -> [B] -> [C] -> None
```

여기서 `HEAD` 는 첫 번째 노드 `[A]` 를 가리킨다.

`[A]` 의 `next` 는 `[B]` 를 가리키고, `[B]` 의 `next` 는 `[C]` 를 가리킨다.

마지막 노드 `[C]` 의 `next` 는 더 이상 다음 노드가 없으므로 `None` 이다.

구현해야 할 함수 목록

`linked_list.skeleton.py` 에서 구현해야 할 주요 함수는 다음과 같다.

- `Node.__init__(value)`
- `LinkedList.__init__()`
- `append(value)`
- `prepend(value)`
- `insert(index, value)`
- `delete(value)`
- `find(value)`
- `get(index)`

- `get_next(index)`

`length()` 와 `display()` 는 skeleton 코드에 이미 구현되어 있으므로, 스터디에서는 보조 함수로 사용하면 된다.

Node.init(value)

사용 목적

`Node` 클래스는 링크드리스트의 노드 하나를 나타낸다.

하나의 노드는 값을 저장하고, 다음 노드를 가리켜야 한다.

구현해야 할 내용

`Node.__init__`에서는 다음 두 가지를 초기화해야 한다.

```
self.value = value
self.next = None
```

`self.value`에는 노드가 가진 실제 값을 저장

`self.next`에는 다음 노드를 저장합니다. 처음 노드를 만들 때는 아직 다음 노드가 연결되어 있지 않기 때문에 `None`으로 초기화

구현 후 기대 동작

```
node = Node("A")

print(node.value)  # A
print(node.next)   # None
```

LinkedList.init()

사용 목적

`LinkedList` 클래스는 링크드리스트 전체를 관리한다.

링크드리스트에서 가장 중요한 시작점은 첫 번째 노드이다.

이 첫 번째 노드를 `head`라고 부른다.

구현해야 할 내용

처음 링크드리스트를 만들면 아무 노드도 없으므로 `head`는 `None`이다.

```
self.head = None
```

구현 후 기대 동작

```
linked_list = LinkedList()

print(linked_list.head)  # None
```

append(value)

사용 목적

`append` 함수는 링크드리스트의 맨 뒤에 새 값을 추가하는 함수이다.

예를 들어 현재 리스트가 다음과 같을 때,

```
HEAD -> [A] -> [B] -> None
```

`append("C")` 를 실행하면 다음과 같이 출력된다.

```
HEAD -> [A] -> [B] -> [C] -> None
```

구현 흐름

1. `value` 를 가진 새 노드를 만든다.
2. 리스트가 비어 있는지 확인한다.
3. 비어 있다면 새 노드를 `head` 로 설정한다.
4. 비어 있지 않다면 `head` 부터 마지막 노드까지 이동한다.
5. 마지막 노드의 `next` 가 새 노드를 가리키도록 연결한다.

핵심 포인트

마지막 노드는 `current.next is None` 인 노드이다.

따라서 마지막 노드를 찾을 때는 다음과 같은 방식으로 이동한다.

```
current = self.head

while current.next is not None:
    current = current.next
```

prepend(value)

사용 목적

`prepend` 함수는 링크드리스트의 맨 앞에 새 값을 추가한다.

예를 들어 현재 리스트가 다음과 같을 때,

```
HEAD -> [B] -> [C] -> None
```

`prepend("A")` 를 실행하면 다음과 같이 출력된다.

```
HEAD -> [A] -> [B] -> [C] -> None
```

구현 흐름

1. `value` 를 가진 새 노드를 만든다.
2. 새 노드의 `next` 가 기존 `head` 를 가리킨다.
3. `self.head` 를 새 노드로 변경한다.

핵심 포인트

순서가 중요하다.

먼저 새 노드가 기존 `head` 를 가리켜야 한다.

그 다음 `head` 를 새 노드로 바꿔야 기존 리스트 연결이 안끊어진다.

```
new_node.next = self.head
self.head = new_node
```

insert(index, value)

사용 목적

`insert` 함수는 원하는 인덱스 위치에 새 값을 추가한다.

예를 들어 현재 리스트가 다음과 같을 때,

```
HEAD -> [A] -> [C] -> None
```

`insert(1, "B")` 를 실행하면 다음과 같이 출력된다.

```
HEAD -> [A] -> [B] -> [C] -> None
```

반환 값

`insert` 함수는 삽입 성공 여부를 반환하는 함수

- 삽입 성공: `True`
- 삽입 실패: `False`

구현 흐름

1. `index` 가 0보다 작으면 `False` 를 반환
2. `index` 가 0이면 `prepend(value)` 를 사용해 맨 앞에 추가하고 `True` 를 반환
3. 새 노드를 만든다.
4. `head` 부터 시작해 삽입할 위치 바로 앞 노드까지 이동
5. 이동 중 `current` 가 `None` 이 되면 인덱스가 너무 큰 것이므로 `False` 를 반환
6. 새 노드를 `current` 와 `current.next` 사이에 연결
7. 삽입에 성공하면 `True` 를 반환

핵심 포인트

인덱스 `2` 에 값을 넣으려면 인덱스 `1` 위치의 노드까지 이동해야 한다.

그래야 새 노드를 그 뒤에 연결할 수 있다.

연결 순서는 다음과 같다.

```
new_node.next = current.next
current.next = new_node
```

먼저 `new_node.next` 를 연결하고, 그 다음 `current.next` 를 바꿔야 기존 뒤쪽 노드를 잃어버리지 않는다.

delete(value)

사용 목적

`delete` 함수는 입력한 값을 가진 첫 번째 노드를 삭제한다.

예를 들어 현재 리스트가 다음과 같을 때,

```
HEAD -> [A] -> [B] -> [C] -> None
```

`delete("B")` 를 실행하면 다음과 같이 출력된다.

```
HEAD -> [A] -> [C] -> None
```

반환 값

`delete` 함수는 삭제 성공 여부를 반환

- 삭제 성공: `True`
- 삭제할 값을 찾지 못함: `False`

구현 흐름

- 리스트가 비어 있으면 `False` 를 반환
- 삭제할 값이 `head` 에 있으면 `head` 를 다음 노드로 옮기고 `True` 를 반환
- 그 외에는 `current.next` 를 확인하며 삭제할 값을 찾는다.
- `current.next.value` 가 삭제할 값과 같으면 `current.next` 를 `current.next.next` 로 바꾼다.
- 삭제에 성공하면 `True` 를 반환
- 끝까지 찾지 못하면 `False` 를 반환

핵심 포인트

중간 노드를 삭제하려면 삭제할 노드의 이전 노드가 필요하다.

그래서 `current.value` 가 아니라 `current.next.value` 를 확인한다.

```
current.next = current.next.next
```

이 코드는 현재 노드가 삭제할 노드를 건너뛰고, 삭제할 노드의 다음 노드를 가리키게 만든다.

find(value)

사용 목적

`find` 함수는 입력한 값이 링크드리스트의 몇 번째 인덱스에 있는지 찾는 함수

예를 들어 현재 리스트가 다음과 같을 때,

```
HEAD -> [A] -> [B] -> [C] -> None
```

`find("B")` 는 1 을 반환

반환 값

- 값을 찾은 경우: 해당 인덱스

- 값을 찾지 못한 경우: `-1`

구현 흐름

1. `current` 를 `head` 로 설정
2. `index` 를 0으로 설정
3. `current` 가 `None` 이 될 때까지 반복
4. `current.value` 가 찾는 값과 같으면 `index` 를 반환
5. 값이 다르면 다음 노드로 이동하고 `index` 를 1 증가시킨다.
6. 끝까지 찾지 못하면 `-1` 을 반환

get(index)

사용 목적

`get` 함수는 입력한 인덱스에 있는 노드의 값을 반환하는 함수

예를 들어 현재 리스트가 다음과 같을 때,

```
HEAD -> [A] -> [B] -> [C] -> None
```

`get(1)` 은 "B" 를 반환

반환 값

- 유효한 인덱스: 해당 노드의 값
- 유효하지 않은 인덱스: `None`

구현 흐름

1. `index` 가 0보다 작으면 `None` 을 반환
2. `current` 를 `head` 로 설정
3. `current_index` 를 0으로 설정
4. `current` 가 `None` 이 될 때까지 반복
5. `current_index` 가 `index` 와 같으면 `current.value` 를 반환
6. 아직 원하는 위치가 아니면 다음 노드로 이동하고 `current_index` 를 1 증가시킨다.
7. 끝까지 찾지 못하면 `None` 을 반환

get_next(index)

사용 목적

`get_next` 함수는 입력한 인덱스에 있는 노드가 가리키는 다음 노드의 참조(reference)를 반환하는 함수

값이 아니라 노드 객체 자체를 반환한다는 점이 중요하다.

예를 들어 현재 리스트가 다음과 같을 때,

```
HEAD -> [A] -> [B] -> [C] -> None
```

`get_next(0)` 은 `[B]` 노드 객체를 반환

`get_next(2)` 는 마지막 노드 `[C]` 의 다음 노드가 없으므로 `None` 을 반환

반환 값

- 다음 노드가 있는 경우: 다음 `Node` 객체
- 마지막 노드이거나 유효하지 않은 인덱스인 경우: `None`

구현 흐름

1. `index` 가 0보다 작으면 `None` 을 반환
2. `current` 를 `head` 로 설정
3. `current_index` 를 0으로 설정
4. `current` 가 `None` 이 될 때까지 반복
5. `current_index` 가 `index` 와 같으면 `current.next` 를 반환
6. 아직 원하는 위치가 아니면 다음 노드로 이동하고 `current_index` 를 1 증가시킨다.
7. 끝까지 찾지 못하면 `None` 을 반환

제공된 보조 함수

`linked_list.skeleton.py` 에서 `length()` 와 `display()` 는 이미 구현되어 있습니다.

length()

`length()` 는 현재 링크드리스트에 들어 있는 노드 개수를 반환하는 함수

```
linked_list.length()
```

예를 들어 노드가 3개라면 `3` 을 반환

display()

`display()` 는 현재 링크드리스트를 터미널에서 보기 좋은 문자열로 반환하는 함수

```
linked_list.display()
```

예를 들어 리스트에 `A`, `B`, `C` 가 들어 있다면 다음과 같은 문자열을 반환

```
HEAD -> [A] -> [B] -> [C] -> None
```

main.py 함수 설명

`main.py` 는 구현한 링크드리스트를 터미널에서 직접 테스트하기 위한 실행 파일이다.

`main.py` 는 다음 코드로 `linked_list.py` 의 `LinkedList` 클래스를 가져온다.

```
from linked_list import LinkedList
```

스터디에서 `linked_list_skeleton.py` 를 테스트하려면 파일 이름이 `linked_list.py` 로 되어 있어야 합니다.

즉, `main.py` 는 기본적으로 `linked_list.py` 라는 파일을 import합니다.

print_menu()

사용 목적

터미널에 사용 가능한 메뉴를 출력하는 함수

메뉴 목록

1. `append` : 맨 뒤에 노드 추가
2. `prepend` : 맨 앞에 노드 추가
3. `insert` : 원하는 인덱스에 노드 추가
4. `delete` : 값으로 노드 삭제
5. `find` : 값으로 노드 검색
6. `get` : 인덱스로 노드 정보 조회
7. `show` : 링크드리스트 출력
0. `exit` : 종료

이 함수는 직접 링크드리스트를 수정하지 않고, 사용자에게 메뉴만 보여준다.

`input_insert(linked_list)`

사용 목적

사용자에게 삽입할 인덱스와 값을 입력받고, `linked_list.insert(index, value)` 를 호출하는 함수

이용 방법

터미널 메뉴에서 `3` 또는 `insert` 를 입력하면 실행된다.

```
명령어를 입력하세요: 3
추가할 인덱스를 입력하세요: 1
추가할 값을 입력하세요: B
```

인덱스가 정수가 아니거나 현재 리스트에 삽입할 수 없는 위치라면 다시 입력받는다.

예를 들어 현재 노드가 2개라면 삽입 가능한 인덱스는 `0`, `1`, `2` 이다.

`input_get(linked_list)`

사용 목적

사용자에게 조회할 인덱스를 입력받고, 해당 위치의 값과 다음 노드 참조를 출력하는 함수

내부적으로 다음 두 함수를 사용

```
linked_list.get(index)
linked_list.get_next(index)
```

이용 방법

터미널 메뉴에서 `6` 또는 `get` 을 입력하면 실행된다.

```
명령어를 입력하세요: 6
조회할 인덱스를 입력하세요: 1
```

조회 가능한 인덱스라면 해당 인덱스의 값과 다음 노드 정보를 출력

마지막 노드라면 다음 노드가 없으므로 `None` 이라고 출력

main()

사용 목적

프로그램 전체 실행을 담당합니다.

동작 흐름

1. `LinkedList()` 객체를 생성
2. 메뉴를 출력
3. 현재 링크드리스트 상태를 출력
4. 사용자 명령어를 입력받는다.
5. 명령어에 맞는 함수를 실행
6. 사용자가 `0`, `exit`, `quit`, `q` 중 하나를 입력할 때까지 반복

main.py 실행 방법

터미널에서 `main.py`가 있는 폴더로 이동합니다.

```
ex) cd /Users/song-jeong-keun/Desktop/KDT/과제/LinkedList
```

그 다음 `main.py`를 실행합니다.

```
python3 main.py
```

실행 예시

아래는 `main.py`를 실행한 뒤 링크드리스트에 값을 추가하고 조회하는 예시입니다.

```
=== Linked List Terminal Viewer ===
1. append   : 맨 뒤에 노드 추가
2. prepend  : 맨 앞에 노드 추가
3. insert   : 원하는 인덱스에 노드 추가
4. delete   : 값으로 노드 삭제
5. find     : 값으로 노드 검색
6. get      : 인덱스로 노드 정보 조회
7. show     : 링크드리스트 출력
0. exit     : 종료

HEAD -> None

# append
명령어를 입력하세요: 1          # 입력 부분
추가할 값을 입력하세요: A       # 입력 부분
'A' 값을 맨 뒤에 추가했습니다.

명령어를 입력하세요: 1          # 입력 부분
추가할 값을 입력하세요: C       # 입력 부분
'C' 값을 맨 뒤에 추가했습니다.

# insert
```

```

명령어를 입력하세요: 3          # 입력 부분
추가할 인덱스를 입력하세요: 1    # 입력 부분
추가할 값을 입력하세요: B
1번 인덱스에 'B' 값을 추가했습니다.

# show
명령어를 입력하세요: 7          # 입력 부분
HEAD -> [A] -> [B] -> [C] -> None

# find
명령어를 입력하세요: 5          # 입력 부분
검색할 값을 입력하세요: B      # 입력 부분
'B' 값은 1번 인덱스에 있습니다.

# get, get_next
명령어를 입력하세요: 6          # 입력 부분
조회할 인덱스를 입력하세요: 1  # 입력 부분
1번 인덱스의 값은 'B'입니다.
1번 인덱스가 갖고있는 reference는 '<linked_list.Node object at ...>'입니다.

# delete
명령어를 입력하세요: 4          # 입력 부분
삭제할 값을 입력하세요: B      # 입력 부분
'B' 값을 삭제했습니다.

# show
명령어를 입력하세요: 7          # 입력 부분
HEAD -> [A] -> [C] -> None

# 종료
명령어를 입력하세요: 0          # 입력 부분
프로그램을 종료합니다.

```

스터디 진행 추천 순서

처음부터 모든 함수를 한 번에 구현하기보다 다음 순서로 구현하면 이해하기 쉬울거라 생각한다.

1. `Node.__init__`
2. `LinkedList.__init__`
3. `append`
4. `display`로 결과 확인
5. `prepend`
6. `insert`
7. `find`
8. `get`
9. `get_next`
10. `delete`

특히 `insert`와 `delete`는 노드 연결 순서가 중요하므로 그림을 그리면서 구현하면 훨씬 이해하기 쉬울거라 생각한다.